

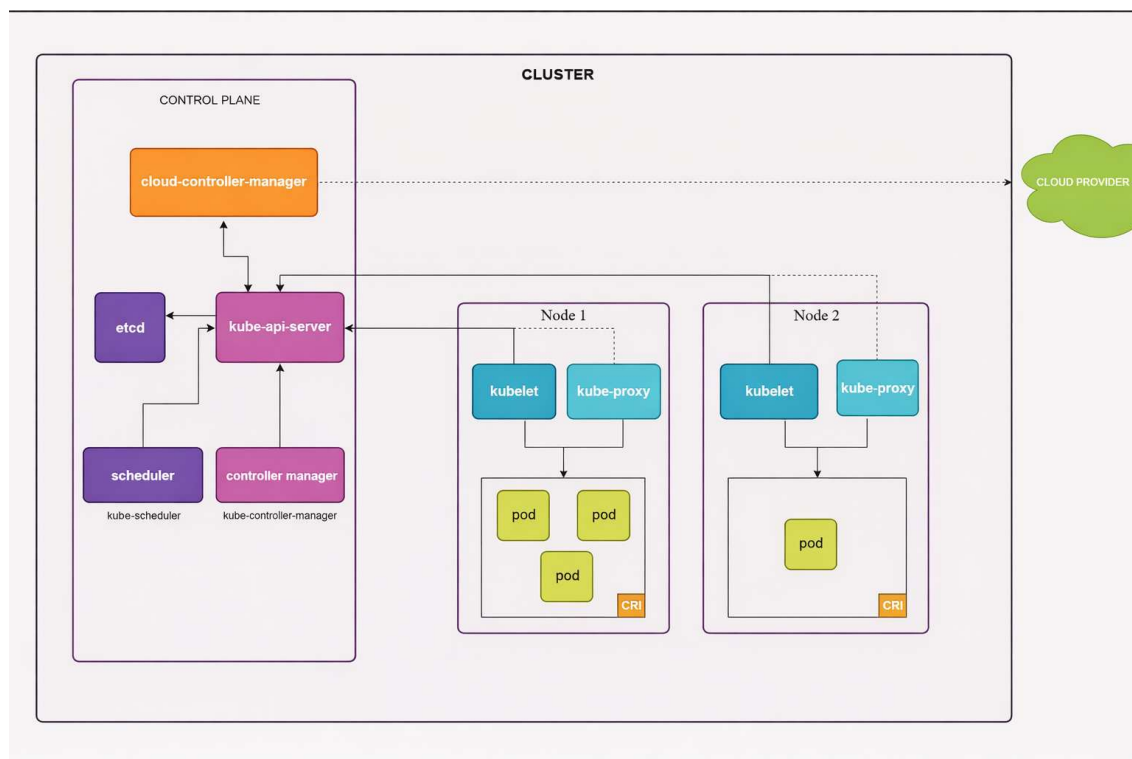
Kubernetes

Kubernetes is a portable, extensible, open source platform for managing containerized workloads and services that facilitate both declarative configuration and automation. It has a large, rapidly growing ecosystem. Kubernetes services, support, and tools are widely available.

The name Kubernetes originates from Greek, meaning helmsman or pilot. K8s as an abbreviation results from counting the eight letters between the "K" and the "s". Google open sourced the Kubernetes project in 2014. Kubernetes combines [over 15 years of Google's experience](#) running production workloads at scale with best-of-breed ideas and practices from the community.

Cluster

A Kubernetes cluster consists of a control plane plus a set of worker machines, called nodes, that run containerized applications. Every cluster needs at least one worker node in order to run Pods.



The worker node(s) host the Pods that are the components of the application workload. The control plane manages the worker nodes and the Pods in the cluster. In production environments, the control plane usually runs across multiple computers and a cluster usually runs multiple nodes, providing fault-tolerance and high availability.

Kubernetes is designed to run and manage containerized applications across a group of machines called a cluster. The cluster is divided into two main parts: the Control Plane and the Worker Nodes. Each part has a clear responsibility, and together they ensure applications run reliably and at scale.

1. Control Plane – The Brain of the Cluster

The control plane is responsible for managing the cluster. It does not run application containers. Instead, it decides *what should run, where it should run, and how to keep it running*.

kube-apiserver

The API server is the central communication hub of Kubernetes. All requests—whether from users, nodes, or internal components—go through the API server.

When you run a command like `kubectl apply`, it is sent to the API server. The API server validates the request and stores the desired state in etcd.

etcd

etcd is the cluster's database.

It stores everything about the cluster: nodes, pods, configurations, secrets, and current state.

Kubernetes relies on etcd to know what the cluster should look like, so keeping etcd healthy and backed up is critical.

kube-scheduler

The scheduler decides which node should run a Pod.

When a Pod is created without a node assignment, the scheduler:

- Looks at available nodes
- Checks CPU, memory, and constraints
- Selects the best node
- Updates the API server with the decision

kube-controller-manager

The controller manager ensures the cluster's actual state matches the desired state.

Examples:

- If a node goes down, it detects it and reacts
- If a Deployment needs 3 Pods but only 2 are running, it creates another
- If a Job must complete, it ensures Pods run until completion

Controllers continuously watch the cluster and automatically fix problems.

cloud-controller-manager (cloud environments only)

This component connects Kubernetes to the **cloud provider**.

It manages:

- Cloud load balancers for Services
- Cloud routes

- Cloud-based node lifecycle

This component is not used in on-premise or local clusters.

2. Worker Nodes – Where Applications Run

Worker nodes are machines that **run application workloads**. Each node has components that communicate with the control plane and manage containers locally.

kubelet

The kubelet is the main agent on every node.

It:

- Receives Pod instructions from the API server
- Talks to the container runtime
- Ensures containers are running as defined
- Reports health and status back to the control plane

The kubelet does not create containers on its own—it only runs what Kubernetes tells it to.

kube-proxy

kube-proxy handles network communication for Services.

It:

- Sets up network rules on the node
- Ensures traffic reaches the correct Pods
- Enables communication inside and outside the cluster

Some modern network plugins replace kube-proxy functionality.

Container Runtime

The container runtime actually runs containers.

Examples:

- containerd
- CRI-O

Kubernetes communicates with the runtime using the Container Runtime Interface (CRI).

3. Add-ons – Supporting Cluster Features

Add-ons provide important cluster-level features and usually run in the kube-system namespace.

Common add-ons include:

- **DNS:** Allows Pods to discover Services by name (essential)
- **Dashboard:** Web interface for cluster management
- **Monitoring:** Collects resource metrics
- **Logging:** Centralizes container logs
- **Network Plugins (CNI):** Enable Pod-to-Pod communication and IP management

4. What Actually Happens When You Deploy an App

1. You submit a Deployment using `kubectl`
2. The request goes to **kube-apiserver**
3. The desired state is stored in **etcd**
4. **Controller manager** creates Pods if needed
5. **Scheduler** assigns Pods to nodes
6. **kubelet** on each node starts containers
7. **kube-proxy** enables network access
8. Kubernetes continuously monitors and self-heals

5. Deployment Models

Kubernetes can be deployed in different ways:

- Single-machine or multi-machine control plane
- Control plane as system services or static Pods
- Self-hosted control plane
- Fully managed Kubernetes from cloud providers

The core architecture remains the same.

Summary

Kubernetes works by **separating decision-making from execution**:

- The **control plane** decides what should happen
- The **nodes** make it happen
- Controllers continuously monitor and fix issues

This design allows Kubernetes to scale, recover from failures, and run applications reliably across many machines.